AI ENGINEERING JOURNEY

WEEK 1 | DAY 1 | FOUNDATIONS | JUL 6 2026

Tokenization

How Large Language Models See Text

The Big Question: When you type "Hello, how are you?" into ChatGPT — what does the model actually see? The answer is: **not letters, not words — tokens.**

Week 1 — Full Day Plan

Day	Date	Topic	Status
Day 1	Mon Jul 6	**Tokenization**	**YOU ARE HERE**
Day 2	Tue Jul 7	Embeddings	Upcoming
Day 3	Wed Jul 8	Transformer Architecture	Upcoming
Day 4	Thu Jul 9	Training Lifecycle	Upcoming
Day 5	Fri Jul 10	Scaling Laws + Prompt Engineering	Upcoming
Day 6	Sat Jul 11	Structured Output + Architect's Lens	Upcoming
Day 7	Sun Jul 12	Week 1 Project — Model Selection Dashboard	Upcoming

Topics Covered Today

#	Topic	Coverage
1	**What is a Token?**	Plain English explanation with real examples
2	**BPE Algorithm**	Step-by-step: how vocabulary is built from training data
3	**WordPiece vs SentencePiece**	Comparison and when to use each
4	**Context Windows**	The model's working memory — limits and implications
5	**Tokens = Money**	Cost model, Netflix FAANG-scale example, $180M cost difference

6	**Architect's Lens**	3 production decisions: multilinguality, counting, output control
7	**Special Tokens**	[CLS], [SEP], endoftext, im_start — what they are and why they matter
8	**Tokenization Quirks**	Leading spaces, numbers, code, emoji — production gotchas
9	**Hands-On Exercise**	Python tiktoken exercise — see tokens with your own eyes

Section 1 — What is a Token? (Plain English)

An LLM does not read text the way you do. It never sees letters or words. It sees **numbers** — specifically, integer IDs that map to chunks of text called **tokens**.

> You have a library of 100,000 "word pieces" (the vocabulary). Every piece has an ID number. When you send text to a model, a **tokenizer** converts your text into a sequence of those ID numbers. The model processes the numbers, produces output numbers, and the tokenizer converts those back into text.

Example — the sentence "Netflix uses AI":

```
"Netflix" → [45431] (1 token)
" uses" → [4773] (1 token)
" AI" → [9552] (1 token)

Full sequence: [45431, 4773, 9552] → 3 tokens, 3 words
```

> **Key Insight:** 1 word is NOT equal to 1 token. English averages **1.3 tokens per word**. Code and non-English text can be 2-3 tokens per word. This directly affects your API cost and context window limits.

This is why the model never encounters a truly 'unknown' word — even the most unusual word can be decomposed into known subword tokens from its vocabulary.

Section 2 — The Three Tokenizer Algorithms

Algorithm	Used By	Key Idea
BPE (Byte Pair Encoding)	GPT-4, LLaMA, Mistral	Merge most-frequent character pairs repeatedly until vocabulary is full
WordPiece	BERT, DistilBERT	Like BPE but uses likelihood scoring instead of raw frequency; keeps word boundaries
SentencePiece	LLaMA-2, T5, Gemma	Language-agnostic; works at byte level; handles Chinese, Japanese, Hindi without word boundaries

> For your daily work, **you will mostly encounter BPE** — OpenAI, Meta, and Mistral all use it. **SentencePiece is the one to reach for when building multilingual systems** — it handles any script out of the box because it works at the raw byte level, not the word level.

Section 3 — How BPE Works: Step by Step

BPE (Byte Pair Encoding) builds its vocabulary by scanning through billions of training words and repeatedly merging the most frequently occurring character pairs. Here is exactly how it works:

Step 1 — Start with individual characters

Every word is initially split into individual characters. This is the starting point before any merging.

```
"unhappy" → [u] [n] [h] [a] [p] [p] [y]
```

Step 2 — Count the most frequent character pairs in training data

Scan through billions of words. Find which pairs of characters appear together most often across the entire training corpus.

```
Most common pair: "t" + "h" → appears 2.3 billion times
Next: "e" + "r" → appears 1.9 billion times
Next: "i" + "n" → appears 1.7 billion times
```

Step 3 — Merge the most frequent pair into a single token

The pair "t"+"h" becomes "th". Repeat this merge process 50,000 times. Each iteration creates a new token in the vocabulary.

```
Round 1: "unhappy" → [u] [n] [h] [a] [pp] [y] ("pp" merged)
Round 2: "unhappy" → [un] [h] [a] [pp] [y] ("un" merged)
Round 3: "unhappy" → [un] [happ] [y] ("happ" merged)
Final: "unhappy" → [un] [happy] (full subwords)
```

Step 4 — The result: a vocabulary of 50,000-100,000 tokens

Common words become single tokens. Rare words are split into subwords. Unknown words are handled by falling back to characters. This is why the model never encounters a truly 'unknown' word.

```
"tokenization" → ["token", "ization"] → [3947, 2065]
"Netflix" → ["Netflix"] → [45431]
"AI" → ["AI"] → [7890] (single token!)
"antidisestablishmentarianism" → 7 tokens
```

Section 4 — Context Windows: The Model's Working Memory

> The context window is the maximum number of tokens the model can "see" at once — both your input AND its output combined. Think of it as the model's RAM.

Definition:

```
Context Window = Input Tokens + Output Tokens
| |
(your prompt + (model response)
documents +
chat history)
```

Current Model Context Limits:

Model	Context Window	Practical Equivalent	Best For
GPT-4o	128K tokens	~300 pages of text	Most enterprise use cases
Claude Sonnet 4	200K tokens	~500 pages of text	Long document analysis
LLaMA-3 8B	8K tokens	~20 pages of text	Self-hosted, cost-sensitive
Gemini 1.5 Pro	1M tokens	~2,500 pages of text	Entire codebases

Why This Matters for Architecture:

RAG Systems (Week 3-4): A 512-token chunk uses 0.4% of GPT-4o's context but 6.4% of LLaMA-3 8B's context. That changes how many chunks you can retrieve per query and directly impacts your retrieval architecture.

Chat Applications: Every previous turn accumulates in the context. At 128K tokens with 500-token messages, you get ~256 turns before the model truncates old messages and starts losing context.

Cost Control: Larger context windows mean higher per-call costs. A 200K token call costs ~8x more than a 25K token call. Context window management is a first-class cost-engineering concern.

Section 5 — Tokens = Money: The Architect's Cost Model

Every token processed by an API model costs money. This is not a footnote — it is a **first-class architectural concern.**

```
Daily cost = (requests/day) x (avg tokens/request) x (price per token)
```

Current Model Pricing:

Model	Input (per 1M tokens)	Output (per 1M tokens)	1M req/day x 500 tokens
GPT-4o	$2.50	$10.00	~$3,750/day
Claude Sonnet 4	$3.00	$15.00	~$4,500/day
GPT-4o mini	$0.15	$0.60	~$225/day
LLaMA-3 8B (self-hosted)	~$0.05	~$0.05	~$75/day (GPU cost)

Architect's Rule: Output tokens cost 3-5x more than input tokens. Always optimize your prompt to get shorter, structured responses.

Real-World FAANG-Scale Example — Netflix AI Recommendation Assistant:

Netflix builds an AI assistant where 50M daily active users each trigger 3 AI calls/day (recommendation explanations, search enhancement, content summaries). Each request uses 600 input tokens and produces 200 output tokens.

```
Input tokens/day = 50,000,000 x 3 x 600 = 90,000,000,000 tokens (90B)
Output tokens/day = 50,000,000 x 3 x 200 = 30,000,000,000 tokens (30B)

Using GPT-4o ($2.50 input / $10.00 output per 1M tokens):
Input cost = 90B x $2.50/1M = $225,000/day
Output cost = 30B x $10.00/1M = $300,000/day
TOTAL = $525,000/day = $191,625,000/year

Switch to GPT-4o-mini ($0.15 input / $0.60 output per 1M tokens):
Input cost = 90B x $0.15/1M = $13,500/day
Output cost = 30B x $0.60/1M = $18,000/day
TOTAL = $31,500/day = $11,497,500/year

SAVING: $180,127,500/year from model selection alone
```

$180M/year saved from one architecture decision. This is why FAANG AI teams have dedicated model selection and cost engineering roles. Output tokens cost 3-5x more than input tokens — always optimise for shorter, structured responses.

This is why as an architect you always ask: What is the minimum model capability that satisfies the task? You do not use GPT-4o for tasks that GPT-4o-mini can handle.

Section 6 — Architect's Lens: 3 Tokenization Decisions in Production

Decision 1 — Tokenizer Choice Affects Multilinguality

If your system will handle Hindi, Arabic, Chinese, or Japanese — choose a model using SentencePiece (LLaMA, Gemma, T5) or verify that your BPE-based model has those scripts well-represented in its vocabulary.

A poorly-tokenized language can use 4-5 tokens per character, exploding your costs and hitting context limits faster. Example: Japanese text in a BPE model trained primarily on English data may use 3-5x more tokens than the same content in English.

Decision 2 — Token Counting Must Be a First-Class Concern in Your Input Pipeline

Before you send anything to an LLM API, count tokens. Every production system needs a pre-flight check: if this input exceeds the context window, truncate / chunk / summarize before sending. Never let the API reject your call at runtime.

Pattern you will use in every production AI system:

```
import tiktoken

enc = tiktoken.encoding_for_model('gpt-4o')
token_count = len(enc.encode(my_text))

if token_count > MAX_INPUT_TOKENS:
# truncate or chunk before sending
my_text = smart_truncate(my_text, MAX_INPUT_TOKENS)
```

Decision 3 — Output Token Limits Are a Cost Lever

Output tokens cost 3-5x more than input tokens. Always constrain your output: max_tokens=500 for summaries, structured JSON responses to avoid verbose prose, few-shot examples in the prompt to enforce concise formatting.

This single change alone can cut output costs by 40-60% with no loss in quality.

Section 7 — Special Tokens

Every model has **special tokens** — reserved tokens with specific roles that never come from your text. You will see these constantly in production logs, fine-tuning code, and debugging sessions.

Token	Model	Purpose
<|endoftext|>	GPT family	Marks the end of a document in training and inference
<|im_start|>	ChatML format	Start of a chat message turn (used by OpenAI chat format)
<|im_end|>	ChatML format	End of a chat message turn
[CLS]	BERT	"Start of sequence" — used as the classification representation

[SEP]	BERT	Separator between two sentences in a pair task
[PAD]	All models	Padding token to make batches equal length during training
<s>	LLaMA, Mistral	Start of sequence marker
</s>	LLaMA, Mistral	End of sequence marker — signals model to stop generating

Why This Matters: When you fine-tune a model, you must format your training data with the exact special tokens that model expects. Use the wrong format and the model produces garbage output. In Week 2 (Fine-Tuning) you will apply this directly — every fine-tuning dataset must be wrapped with the correct special token structure.

Section 8 — Tokenization Quirks That Bite You in Production

These are things that confuse even experienced engineers. Knowing them in advance will save you hours of debugging in production.

Quirk 1 — Leading Space Changes the Token

The same word gets different token IDs depending on whether it starts a sentence. This matters when you are doing exact token-level matching or fine-tuning where position in a sentence changes the token representation.

```
enc.encode("Apple") → [24895] # 1 token
enc.encode(" Apple") → [15508] # 1 token, but DIFFERENT ID
# The space before the word is baked into the token
```

Quirk 2 — Numbers Tokenize Digit by Digit

This is why LLMs are notoriously bad at arithmetic — they never 'see' a whole number, only subword pieces. An LLM doing math is like you doing addition while looking at individual letters of each number.

```
enc.encode("2024") → [2518, 19] # "20" + "24" = 2 tokens
enc.encode("12345") → [4364, 1774] # "123" + "45" = 2 tokens
enc.encode("$1,000,000") → 7 tokens
# Every symbol and digit chunk splits separately
```

Quirk 3 — Code Is Token-Expensive

A 500-line Python file can easily be 3,000-5,000 tokens. If you are building a code review agent, this is your primary sizing constraint — you may only fit a fraction of a codebase in a single context window.

```
enc.encode("def calculate_total(items: list) -> float:")
→ 14 tokens for a single function signature

# Underscores, brackets, colons, type hints all tokenize separately
# A 500-line Python file = 3,000-5,000 tokens
```

Quirk 4 — Emoji and Unicode Bloat

Emoji are multi-byte UTF-8 characters. Each byte can be a separate token, meaning a single emoji may cost 3 tokens. For multilingual applications with heavy emoji use (social media, customer support) this adds up.

```
enc.encode("■") → 3 tokens # single emoji = 3 tokens!
enc.encode("cafe") → 1 token
enc.encode("café") → 3 tokens # the accent splits the word
# Implication: emoji-heavy text is significantly more expensive
```

Section 9 — Hands-On Exercise: See Tokens With Your Own Eyes

Run this in your terminal. Seeing tokenizer output with your own text makes everything click faster than reading alone.

Setup — Run in Terminal:

```
cd /Users/thaya/AI-Engineering/week-01/notebooks
source /Users/thaya/AI-Engineering/.venv/bin/activate
pip install tiktoken # if not already installed
```

Create file: week-01/notebooks/day1_tokenization.py

```
import tiktoken

enc = tiktoken.encoding_for_model('gpt-4o')

examples = [
'Hello Netflix!',
'tokenization',
"SELECT * FROM employees WHERE department = 'Engineering'",
'def calculate_embeddings(text: str) -> list[float]:',
'AI is transforming enterprise software.',
'[redacted]', # Chinese: same sentence
]

print(f"{'Text':<55} {'Tokens':>6} Token IDs")
print('-' * 90)
for text in examples:
token_ids = enc.encode(text)
token_strs = [enc.decode([t]) for t in token_ids]
print(f'{text:<55} {len(token_ids):>6} {token_strs}')
```

What You Will Observe:

- The SQL query uses many more tokens than the English sentence — punctuation and keywords split into separate tokens
- The Python function signature is also token-heavy — underscores, brackets, colons all tokenize individually
- The Chinese sentence (same meaning) uses roughly 2x the tokens of English — this is a real cost multiplier for multilingual systems
- Run with your own text to build intuition: try SQL queries, Python code, or any non-English language

Day 1 — Complete Summary

#	Topic	Key Takeaway	Status

1	**What is a Token**	Token = integer ID for a chunk of text; LLMs see sequences of numbers not words	**DONE**
2	**BPE Algorithm**	Merge most frequent character pairs from training data; builds 50K-100K token vocabulary	**DONE**
3	**WordPiece vs SentencePiece**	SentencePiece is language-agnostic and byte-level — use it for multilingual systems	**DONE**
4	**Context Windows**	Context = Input + Output tokens combined; GPT-4o 128K, Claude 200K, LLaMA-3 8K	**DONE**
5	**Token Cost Model**	Wrong model = $180M+/year extra cost at Netflix-scale AI; output tokens cost 3-5x more	**DONE**
6	**Architect's Lens**	3 decisions: multilinguality, token counting pre-flight, output constraints	**DONE**
7	**Special Tokens**	[CLS],[SEP],[PAD], endoftext, im_start — use correct format when fine-tuning	**DONE**
8	**Tokenization Quirks**	Leading spaces, number splitting, code is expensive, emoji = 3 tokens each	**DONE**
9	**Hands-On Exercise**	tiktoken Python exercise at week-01/notebooks/day1_tokenization.py	**ACTION**

Preview — Day 2: Embeddings (Tuesday, Jul 7)

Now that you know tokens are integer IDs, the next question is: **how does the model understand meaning?**

The answer is **embeddings** — the model converts each token ID into a high-dimensional vector (a list of 1,536 floating-point numbers for OpenAI's models). These vectors encode semantic meaning: the vector for "king" minus "man" plus "woman" is geometrically close to "queen."

Day 2 will cover:

- What embeddings are and how they encode meaning as geometry
- How similarity search works — cosine similarity and dot product
- Why embeddings are the foundation of RAG, semantic search, and recommendation systems
- How to generate and use embeddings in Python (OpenAI + sentence-transformers)
- Architect's Lens: choosing embedding models for production — dimensions, speed, cost

EAGLE EYE ADDENDUM — DAY 1 — TOKENIZATION

Unigram Language Model Tokenizer

Beyond BPE and WordPiece, there is a third major tokenization algorithm: the **Unigram Language Model** tokenizer, implemented in SentencePiece and used by T5, ALBERT, XLNet, and many multilingual models.

How Unigram Works — Top-Down Pruning

Unlike BPE (which builds vocabulary bottom-up by merging) and WordPiece (which also merges greedily), Unigram works **top-down**: it starts with a large vocabulary and prunes tokens that contribute least to the language model.

Step	BPE	WordPiece	Unigram
Direction	Bottom-up (merge)	Bottom-up (merge)	Top-down (prune)
Start	Character vocabulary	Character vocabulary	Large initial vocab
Criterion	Merge frequency	Likelihood gain	Remove lowest-loss token
Output	Deterministic	Deterministic	Probabilistic
Used by	GPT, LLaMA, Mistral	BERT, RoBERTa	T5, ALBERT, mT5
SentencePiece	Yes (option)	Yes (option)	Yes (default mode)

Unigram in Practice

SENTENCEPIECE UNIGRAM — T5 example

```
import sentencepiece as spm

# T5 uses SentencePiece Unigram model (vocab = 32,000 tokens)
sp = spm.SentencePieceProcessor()
sp.load('spiece.model')  # Google T5 SentencePiece model

# Unigram produces probabilistic segmentation:
# 'tokenization' might be split multiple ways, model picks highest probability
text = 'tokenization'
print(sp.encode(text, out_type=str))
# → ['▁token', 'ization']   (▁ marks word start)

# Why probabilistic matters:
# BPE: always same split for same input
# Unigram: can sample different splits — acts as data augmentation during training
```

Architect Note — When to Choose Unigram

Use Unigram when building multilingual models or when you want stochastic tokenization (sampling different splits) for training robustness. BPE is better for English-focused models. WordPiece for BERT-style MLM pretraining. Unigram for T5-style seq2seq or multilingual.

Vocabulary Size Reference — Key Numbers

Every model architect makes a deliberate vocabulary size choice. These are the landmark numbers every AI engineer must know:

Model	Tokenizer	Vocab Size	Key Decision
GPT-2	tiktoken BPE	50,257	50K is the classic GPT sweet spot
GPT-3 / GPT-4	tiktoken cl100k	100,277	2x GPT-2; better code + multilingual
BERT (base/large)	WordPiece	30,522	Smaller; English-focused
LLaMA 1/2	SentencePiece BPE	32,000	Same as T5, compact
LLaMA 3	tiktoken BPE	128,256	4x GPT-2; best multilingual coverage
T5 / mT5	SentencePiece Unigram	32,000 / 250,000	mT5 covers 101 languages
Gemini	SentencePiece	256,000	Largest; maximises language coverage

FAANG Tokenization Decisions — Google & Amazon

Google — SentencePiece + Unigram Across All Products	
Problem	Google needs one tokenizer that works across Google Search, Gmail, Google Translate (100+ languages), and YouTube.
Solution	Google created SentencePiece (open-sourced 2018) with Unigram LM. T5 uses SentencePiece Unigram with 32,000-token vocab. Gemini uses SentencePiece with 256,000-token vocab for maximum multilingual coverage. Google Translate processes text as raw bytes — no language detection needed.
Architect lesson	Google's choice: larger vocab + Unigram = better multilingual quality. GPT-style BPE optimises for English; SentencePiece Unigram optimises globally.

Amazon — Alexa Tokenization at 500M+ Requests/Day

Problem	Alexa processes natural speech: 'remind me at two thirty', 'order more paper towels', 'what's the weather in Seattle'. Speech has no spaces, contractions, and brand names like 'Tide Pods' that evolve constantly.
Solution	Alexa uses a custom SentencePiece model trained on voice transcripts (not web text). Brand names and product categories get their own tokens. Vocabulary updated quarterly as new products launch. Character-level fallback for truly novel proper nouns.
Architect lesson	Domain-specific tokenizer outperforms generic one for specialised applications. Amazon's 32K-token voice model outperforms GPT-4's 100K-token text model on Alexa-specific tasks.

EAGLE EYE ADDENDUM — Day 1: OOV — Out-of-Vocabulary Problem

Why OOV Matters in Tokenization

Before subword tokenization existed, NLP models used word-level vocabularies. A word-level vocabulary of 100,000 words sounds large — but English has over 170,000 words in common use, and new words (brand names, technical terms, slang, typos, foreign words) appear constantly. Any word not in the vocabulary is Out-of-Vocabulary (OOV). The model had to replace it with a special (unknown) token and lose all semantic information.

Era	Method	OOV Problem	Solution
Pre-2015	Word-level vocab	All new/rare words →	None; OOV is permanent information loss
2015–2018	Character-level	No OOV (chars are finite)	Too slow; loses word-level patterns
2018–now	BPE / WordPiece / SPM	No true OOV; unknown words split to subwords	Rare words = multiple known subword tokens

How BPE Solves OOV

```
# Word-level tokenizer — OOV demonstration
word_vocab = {"the","cat","sat","on","mat"}
sentence   = "the catastrophe was unfathomable"
tokens = [w if w in word_vocab else "<UNK>" for w in sentence.split()]
print(tokens)
# ["the", "<UNK>", "<UNK>", "<UNK>"]
# 3 out of 4 words are OOV — 75% information loss

# BPE tokenizer — NO OOV
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("meta-llama/Llama-3.2-1B")
tokens = tok.tokenize("the catastrophe was unfathomable")
print(tokens)
# ["the", "▁catas", "trophe", "▁was", "▁un", "fath", "omable"]
# Every character is covered; semantic subwords extracted from unknown words
```

The OOV Fallback Chain in Modern Tokenizers

```
Unknown input word → split by BPE merge rules → if subword exists in vocab: use it
                                                → if only single char: byte fallback
                                                  (GPT-2/LLaMA use byte-level BPE)
                                                → <UNK> token (rare; only in older models)

# GPT-2 / LLaMA: byte-level BPE — ZERO OOV POSSIBLE
# Every Unicode character maps to a sequence of bytes (0-255)
# All 256 bytes are in the vocabulary as fallback tokens
# Result: ANY string tokenizable — including emoji, code, math symbols, Chinese
```

```
# Example: rare Unicode character
tok = AutoTokenizer.from_pretrained("gpt2")
print(tok.encode("Hello★World"))
# [15496, 33078, 239, 10603]  ← ★ splits to byte tokens — no UNK
print(tok.decode([15496, 33078, 239, 10603]))
# 'Hello★World'  ← perfect round-trip
```

WHY THIS MATTERS FOR ENGINEERS: When your users send SQL column names like `customer_lifetime_value_usd_2024`, domain acronyms like `EBITDA`, or product codes like `SKU-XZ9-2847` — modern BPE tokenizers handle them gracefully by splitting into known subwords. An older word-level tokenizer would map all of these to and your fine-tuned model would fail. This is the core reason BPE dominates production LLMs.